

ISCM (DefenDoor Windows Hardening Scanner)

Phase 14.6 — Documentation & Handover

Field	Value
Project	ISCM (DefenDoor Windows Hardening Scanner)
Document	Phase 14.6 — Documentation & Handover
Branch	epic/phase14-engine-boundary
Date	2026-09-13 (Shamsi: 1405-06-22)
Test baseline	189 tests, 0 flaky
Source of truth	Code on branch epic/phase14-engine-boundary

PART 1 — PORTABILITY GUIDE

1. Architecture Overview

Phase 14 introduces four cooperating subsystems that form a detection → adaptation → caching → configuration chain. All four are registered as **singletons** in ISCM.Web/Program.cs.



DI registration (from ISCM.Web/Program.cs):

- IEnvironmentDetector → EnvironmentDetector (Singleton)
- IAdaptiveExecutionEngine → AdaptiveExecutionEngine (Singleton)
- IProcessRunner → ProcessRunner (Singleton)
- IProcessCacheService → CachedProcessRunner(runner, config) (Singleton)
- IScannerConfigurationService → ScannerConfigurationService(configuration) (Singleton)

2. Environment Detection (14.1)

2.1 EnvironmentProfile sections

Source: ISCM.Domain/ValueObjects/EnvironmentProfile.cs, ISCM.Infrastructure/Scanning/Collectors/EnvironmentDetector.cs.

Section	Type	Fields
Permissions	PermissionProfile	IsAdmin, CanReadRegistry, CanWriteRegistry, CanRunSecedit, CanRunPowerShell
AvailableTools	ToolAvailabilityProfile	PowerShell, PowerShellVersion, NetExe, Secedit, Auditpol
Hardware	HardwareProfile	ProcessorCount, TotalMemoryMB, IsVirtualMachine, MachineName
PerformanceTier	PerformanceTier enum	Fast, Medium, Slow
Warnings	IReadOnlyList<string>	Human-readable capability warnings
DetectedAt	DateTimeOffset	UTC timestamp of detection

Derived flags on EnvironmentProfile:

- IsFullyCapable = IsAdmin && CanReadRegistry && PowerShell && NetExe
- HasCriticalDeficiencies = !CanReadRegistry || !PowerShell

2.2 VM detection

DetectIsVirtualMachine() inspects two registry locations:

1. BIOS — HKLM\HARDWARE\DESCRIPTION\System\BIOS

Values: SystemManufacturer, SystemProductName, BIOSVersion

Indicators (case-insensitive substring): VMware, VirtualBox, Hyper-V, QEMU, KVM, Xen, Parallels, Virtual Machine, VM

2. Disk — HKLM\SYSTEM\CurrentControlSet\Services\Disk\Enum

Any value containing Virtual, VMware, Or QEMU

2.3 Performance tier rules

From DeterminePerformanceTier(HardwareProfile):

Condition	Tier
ProcessorCount < 2 OR IsVirtualMachine	Slow
ProcessorCount < 4 OR TotalMemoryMB < 4096	Medium
Otherwise	Fast

2.4 Caching

- Result is stored in _cachedProfile under a private lock.
- DetectEnvironmentAsync() returns the cached profile on subsequent calls.
- GetCachedProfile() returns the cached value or null if detection has not run.
- Quick methods (HasPermission, IsToolAvailable, GetPerformanceTier) fall back to lightweight checks when the cache is empty.

2.5 Warnings collected

Condition	Warning text
!IsAdmin	Not running as Administrator - registry writes and remediations will fail
!CanReadRegistry	Cannot read HKLM registry - most checks will fail
!PowerShell	PowerShell not found - PowerShell-based checks will fail
!NetExe	net.exe not found - account-related checks will fail
!Secedit	secedit.exe not found - security policy checks will fail
IsVirtualMachine	Virtual machine detected - some hardware checks may be unreliable
ProcessorCount < 2	Low CPU count ({n}) - scan will run sequentially

3. Adaptive Execution (14.2)

Source: ISCM.Application/Services/AdaptiveExecutionEngine.cs, ISCM.Domain/ValueObjects/ScanExecutionProfile.cs.

3.1 Parallelism rules

Priority: user override > environment tier.

Source	Rule
User override (MaxDegreeOfParallelism > 0)	Use configured value as-is

Source	Rule
Fast tier	Environment.ProcessorCount (or Hardware.ProcessorCount)
Medium tier	max(2, ProcessorCount / 2)
Slow tier	1 (sequential)

3.2 Timeout multiplier

Base values come from `ScannerConfiguration.CheckTimeoutSeconds` and `ParserTimeoutSeconds`.

Tier	Multiplier
Fast	1.0×
Medium	1.5×
Slow	2.5×

Effective timeout = base × multiplier.

3.3 Check-skip table

Hardcoded dictionary `checkToolRequirements` in `AdaptiveExecutionEngine`:

CheckId	Required tool(s)
AUD-001	auditpol.exe (ToolType.Auditpol)
LCK-001	net.exe (ToolType.NetExe)
PWD-001	net.exe (ToolType.NetExe)
URA-001	secedit.exe (ToolType.Secedit)

A check is added to `ChecksToSkip` only when a required tool is unavailable (`IsToolAvailable` returns false).

Note: Missing Administrator is recorded as a **warning only** and never causes a skip:

```
if (!envProfile.Permissions.IsAdmin)
{
    reasoning.Add("Warning: Not running as Administrator - some checks may return Error status");
}
```

Code comment in source:

```
// TODO: Phase 14.3 - Move this to CheckDefinition metadata
```

Note: this migration did NOT happen in Phase 14; it is scheduled as Phase 15 Track C (see PART 3, section 2).

(The dictionary remains hardcoded as of this branch.)

3.4 ScanExecutionProfile fields

Field	Description
EffectiveMaxDegreeOfParallelism	Final parallelism value
EffectiveCheckTimeout	Adjusted check timeout
EffectiveParserTimeout	Adjusted parser timeout
ChecksToSkip	List of CheckIds to skip
Reasoning	Human-readable decision log
ComputedAt	UTC computation timestamp
HasSkippedChecks	ChecksToSkip.Count > 0
IsReducedParallelismMode	EffectiveMaxDegreeOfParallelism == 1

4. Process Caching (14.3)

SOURCES: `CachedProcessRunner.cs`, `ProcessRunner.cs`, `IProcessCacheService.cs`, `ProcessResult.cs`, `ProcessCacheStats.cs`.

4.1 Cache key normalization

```
// NormalizeCacheKey
var normalizedCmd = command?.Trim().ToLowerInvariant() ?? string.Empty;
var normalizedArgs = arguments?.Trim() ?? string.Empty;
return $"{normalizedCmd} {normalizedArgs}";
```

- Command is lowercased and trimmed.
- Arguments are trimmed only (case preserved).

4.2 TTL source

TTL is read from `ScannerConfiguration.CacheMaxAgeMinutes` via `IScannerConfigurationService.GetCacheMaxAge()` → `TimeSpan.FromMinutes(...)`.

Default in `ScannerConfiguration`: **30 minutes**.

4.3 Failed commands never cached

```
// Only cache successful results
if (result.Success) // Success => ExitCode == 0
{
    _cache[cacheKey] = new CacheEntry { ... };
}
```

4.4 Thread-safety

- **Storage:** `ConcurrentDictionary<string, CacheEntry>`
- **Stats:** `Interlocked.Increment / Interlocked.Add` ON `_cacheHits`, `_cacheMisses`, `_totalSavedMs`

4.5 Invalidation

Method	Behavior
<code>Invalidate(commandPattern)</code>	Removes all keys whose normalized form starts with the pattern (case-insensitive)
<code>InvalidateAll()</code>	Clears the entire dictionary
<code>IsCached(command, arguments)</code>	True only if entry exists and age < maxAge

4.6 Current consumers

Verified by reading check constructors:

Check	Usage
<code>AccountLockoutCheck</code>	Injects <code>IProcessCacheService</code> ; calls <code>GetOrRunAsync("net", "accounts")</code> once per collect
<code>AdvancedAuditCheck</code>	Injects <code>IProcessCacheService</code> ; calls <code>GetOrRunAsync("auditpol", "/get /category:*")</code> once per collect

`PasswordLengthCheck` and `UserRightsCheck` still use private `Process.Start` helpers and do **not** consume the cache layer on this branch.

4.7 ProcessResult / ProcessCacheStats

ProcessResult: `Output`, `ErrorOutput`, `ExitCode`, `DurationMs`, `ExecutedAt`, `Command`, `Arguments`; helpers `Success`, `CombinedOutput`, `OutputOnError`.

ProcessCacheStats: `TotalEntries`, `CacheHits`, `CacheMisses`, `TotalSavedMs`, `HitRate`, computed `TotalRequests`.

ProcessRunner defaults: timeout 60 seconds when caller passes `null`; kills process tree on timeout; never throws (errors returned in `ProcessResult` with `ExitCode = -1`).

5. Configuration Profiles (14.4)

Sources: `ISCM.Web/appsettings.json`, `appsettings.Development.json`, `appsettings.Production.json`, `appsettings.AirGapped.json`, `ScannerConfiguration.cs`, `ScannerConfigurationService.cs`.

5.1 Profile values (actual values from files)

Setting	Base (appsettings.json)	Development	Production	AirGapped
<code>CacheMaxAgeMinutes</code>	30	5	60	1440
<code>EnableCache</code>	true	*(inherits)*	true	true
<code>MaxScanDurationMinutes</code>	60	*(inherits)*	120	180
<code>ParserTimeoutSeconds</code>	30	10	60	180
<code>CheckTimeoutSeconds</code>	60	20	120	300

Setting	Base (appsettings.json)	Development	Production	AirGapped
MaxDegreeOfParallelism	0	*(inherits)*	0	1
EnableFingerprintValidation	true	*(inherits)*	true	false
EnableFreshnessPolicy	true	*(inherits)*	true	true
VerboseLogging	false	true	false	false
LogRawOutput	false	true	false	false

Defaults in `ScannerConfiguration` class (used when a key is absent):

`CacheMaxAgeMinutes=30, EnableCache=true, MaxScanDurationMinutes=60, ParserTimeoutSeconds=30, CheckTimeoutSeconds=60, MaxDegreeOfParallelism=0, EnableFingerprintValidation=true, EnableFreshnessPolicy=true, VerboseLogging=false, LogRawOutput=false.`

5.2 Profile selection

ASP.NET Core loads:

- 1. `appsettings.json` (base)
- 2. `appsettings.{ASPNETCORE_ENVIRONMENT}.json` (overrides)

Set environment variable:

```
$env:ASPNETCORE_ENVIRONMENT = "Production" # or Development, AirGapped
```

For `AirGapped`, the environment name must be exactly `AirGapped` so that `appsettings.AirGapped.json` is loaded.

5.3 Invalid-value fallback

`ScannerConfigurationService.BindFromConfiguration()` uses `int.TryParse` / `bool.TryParse`. If parsing fails for a key, that property is left at its current (default or previously bound) value. There is no exception for malformed values.

6. Running on a New Machine

- 1. **Install .NET 8 SDK/Runtime** (Windows x64).
- 2. **Clone / deploy** the application binaries (or build from source: `dotnet build`).
- 3. **Set environment profile** (optional):

```
$env:ASPNETCORE_ENVIRONMENT = "Production" # or Development / AirGapped
```

- 4. **Ensure storage path is writable:**

`%LOCALAPPDATA%\DefenDoor\Data` (from `Storage:RootPath` in base `appsettings`).

- 5. **Run:**

```
dotnet run --project ISCM.Web
```

- 6. **Read startup console:**

- Look for: `Catalog Integrity Validation: PASSED`
- Note any environment warnings emitted by detection (admin, missing tools, VM).

- 7. **Execute a Full scan** from the UI (or via `IScanService`).
- 8. **Verify** findings count matches registered check count and review any skipped checks in the adaptive profile reasoning.

7. Troubleshooting

Symptom	Likely cause	Action
Catalog Integrity Validation: FAILED	Catalog definition issues (duplicates, orphans, missing typed contracts)	Inspect <code>ICatalogValidator</code> output; fix <code>ControlCatalog</code> definitions
Checks return Error / empty evidence for AUD-001	<code>auditpol.exe</code> missing from PATH	Install RSAT / ensure <code>System32</code> is on PATH; confirm <code>AdaptiveExecutionEngine</code> adds AUD-001 to <code>ChecksToSkip</code>
Account lockout / password policy checks fail	<code>net.exe</code> unavailable	Verify <code>net.exe</code> exists under <code>System32</code> ; check environment warnings
User rights checks fail	<code>secedit.exe</code> unavailable or non-admin	Confirm <code>secedit</code> present; run elevated if policy export is required

Symptom	Likely cause	Action
Scan runs sequentially on multi-core host	PerformanceTier = Slow (VM detected or ProcessorCount < 2) or AirGapped profile forces parallelism = 1	Inspect EnvironmentProfile.Hardware.IsVirtualMachine and ASPNETCORE_ENVIRONMENT
Cache never hits	EnableCache=false, TTL expired, or only failed commands executed	Call IProcessCacheService.GetStats(); verify CacheMaxAgeMinutes and successful exit codes
Remediations fail	Not running as Administrator	Restart elevated; Permissions.IsAdmin must be true
SQLite / snapshot errors	Storage path not writable	Ensure %LOCALAPPDATA%\DefenDoor\Data exists and is writable by the process identity

PART 2 — DEPLOYMENT CHECKLIST

1. Pre-deployment requirements

Requirement	Detail
Runtime	.NET 8 (Windows)
OS	Windows 10 / 11 or Windows Server 2019+
Storage	%LOCALAPPDATA%\DefenDoor\Data (SQLite + snapshots) must be writable
Admin	Not required for scan-only; required for registry writes and remediation
Tools	Prefer presence of powershell.exe, net.exe, secedit.exe, auditpol.exe for full coverage

2. Profile selection

Profile	When to use
Development	Local debugging; short timeouts; verbose + raw output logging
Production	Normal production hosts; longer timeouts; cache 60 min; quiet logging
AirGapped	Isolated networks; very long timeouts; sequential execution; fingerprint validation off; cache 24 h

3. Per-profile checklists

Development

- ASPNETCORE_ENVIRONMENT=Development
- Confirm short timeouts (Check=20s, Parser=10s) are acceptable for target machine
- Expect verbose logs and raw output in logs
- Cache TTL = 5 minutes (frequent refresh during development)

Production

- ASPNETCORE_ENVIRONMENT=Production
- Confirm storage path on production volume
- Max scan duration 120 minutes acceptable for host size
- Fingerprint validation and freshness policy enabled
- Logging at Warning/Error only

AirGapped

- ASPNETCORE_ENVIRONMENT=AirGapped
- Confirm **no external network dependency** is required by the application for scanning
- Accept sequential execution (MaxDegreeOfParallelism=1)
- Fingerprint validation disabled by design
- Long timeouts (Check=300s, Parser=180s) and 24 h cache

4. Post-deployment verification

1. Startup log contains:

Catalog Integrity Validation: PASSED

- 2. Run a **Full** scan.
- 3. Findings count equals the number of registered `IShardeningCheck` implementations (22 checks are registered in `Program.cs` on this branch).
- 4. Optional: resolve `IProcessCacheService` and call `GetStats()` after a second scan to confirm cache hits for `net accounts / auditpol`.
- 5. Review adaptive profile reasoning (via `IAdaptiveExecutionEngine.GetCachedProfile()`) for unexpected skips.

5. Rollback procedure

- 1. Stop the running process.
- 2. Redeploy the previous known-good binary set (or checkout previous tag/commit).
- 3. Restore `%LOCALAPPDATA%\DefenDoor\Data` from backup if schema/data changed.
- 4. Set `ASPNETCORE_ENVIRONMENT` back to the prior profile.
- 5. Start and confirm catalog validation passes.

6. Security notes

- Administrator rights are required only when remediation or registry write paths are used. Scan-only operation can run non-elevated (with reduced capability warnings).
- `AirGapped` profile does not introduce external network calls for scanning; fingerprint validation is disabled, which reduces external dependency surface.
- Process execution (`ProcessRunner`) uses `UseShellExecute=false` and `CreateNoWindow=true`; failed commands return structured errors rather than throwing.

PART 3 — PHASE 15 PLANNING

1. Carry-over debt (compiler warnings)

Exact per-project counts are committed in Appendix A of this document (captured 2026-09-13). Phase 15 Track B must re-capture counts after each cleanup commit and update Appendix A accordingly. Categories known from prior architecture notes and code patterns:

Category	Typical location	Notes
CA1416	RemediationService and other Windows-only APIs	Platform-specific; acceptable for Windows-only product; many call sites lack <code>[SupportedOSPlatform("windows")]</code>
CS8618	Domain entities / value objects	Non-nullable properties without definite assignment in constructors
CS8604	Evaluators / parsers	Possible null arguments passed to non-nullable parameters
CS1998	Async methods without <code>await</code>	Some collector helpers marked <code>async</code> but run synchronously
CS8625	WindowsHardeningScanner	Nullable reference assignment / argument issues

Action for Phase 15: run `dotnet build / dotnet build -warnaserror` on a clean machine and capture exact counts per code before closing the cleanup track.

2. Candidate tracks

A. UI/UX Improvements

- Real-time scan progress
- Finding explorer with evidence drill-down
- Remediation workflow UI
- Snapshot comparison viewer

B. Code Quality & Warning Cleanup

- Resolve CA1416, CS8618, CS8604, CS1998, CS8625
- Add missing XML docs on public APIs
- Align nullable annotations with actual contracts

C. Check Metadata System

- Move tool requirements and expected duration out of the hardcoded `CheckToolRequirements` dictionary in `AdaptiveExecutionEngine`
- Store as metadata on check / catalog definitions (addresses the `// TODO: Phase 14.3 - Move this to CheckDefinition metadata comment`)
- Migrate remaining private `Process.Start` consumers (`PasswordLengthCheck`, `UserRightsCheck`, and any other check still using a private `RunCommandAsync` helper) to `IProcessCacheService`, so ALL external command execution flows through the caching layer.

D. Advanced Features (preview)

- Multi-host orchestration
- Scheduled scans
- Trend / history analytics over snapshots

3. Recommended order

B → C → A → D

1. **B first** — Warning cleanup and nullable correctness reduce noise and risk before structural changes. Measurable via zero target warning categories in CI.
2. **C second** — Check metadata unblocks accurate adaptive skip/timeout decisions without hardcoding and prepares the engine for new checks without code changes in `AdaptiveExecutionEngine`.
3. **A third** — UI work benefits from stable engine contracts and clean metadata surfaces.
4. **D last** — Multi-host / scheduling / trending depend on solid core + persistence already delivered in Phase 13 and the adaptive engine from Phase 14.

4. Success criteria per track

Track	Measurable success criteria
B	<code>dotnet build</code> produces 0 warnings in categories CA1416 (annotated), CS8618, CS8604, CS1998, CS8625 for product projects; existing 189 tests remain green
C	<code>CheckToolRequirements</code> dictionary removed from <code>AdaptiveExecutionEngine</code> ; tool requirements and duration read from catalog/check metadata; unit + integration tests cover skip decisions driven by metadata; 189+ tests green
A	UI shows live progress, finding detail with evidence, remediation trigger path, and side-by-side snapshot diff; covered by UI or integration tests where applicable
D	At least one multi-host or scheduled-scan path exists behind a feature flag; snapshot trending query returns deterministic results under test

Appendix A — Exact compiler-warning inventory

Captured from a clean `dotnet build ISCM.sln` log on 2026-09-13

(Shamsi: 1405-06-22), branch `epic/phase14-engine-boundary`:

Project	Count	Categories
ISCM.Domain	10	CS8618 ×10 (CatalogCoverageReport ×7, ControlResult ×2, ScanResult ×1)
ISCM.Application	25	CA1416 ×19 (RemediationService), CS8604 ×4 (ControlEvaluator ×1, TypedEvidenceEvaluator ×1, ComparisonEngine ×2), CS1998 ×2 (EvidenceAcquisitionService)
ISCM.Infrastructure	1	CS8625 ×1 (WindowsHardeningScanner line 162)
ISCM.Tests	1	CS1998 ×1 (ProcessCachingTests line 253)
Total	37	

End of Phase 14.6 Documentation & Handover.

All numeric and behavioral claims above were taken from the listed source files on branch `epic/phase14-engine-boundary`.